

**BUSINESS
PROFESSIONALS**
of **AMERICA**
Giving Purpose to Potential

C# PROGRAMMING

(330)

NATIONAL 2023

PRODUCTION:

NationalBPA

_____ (600 points)

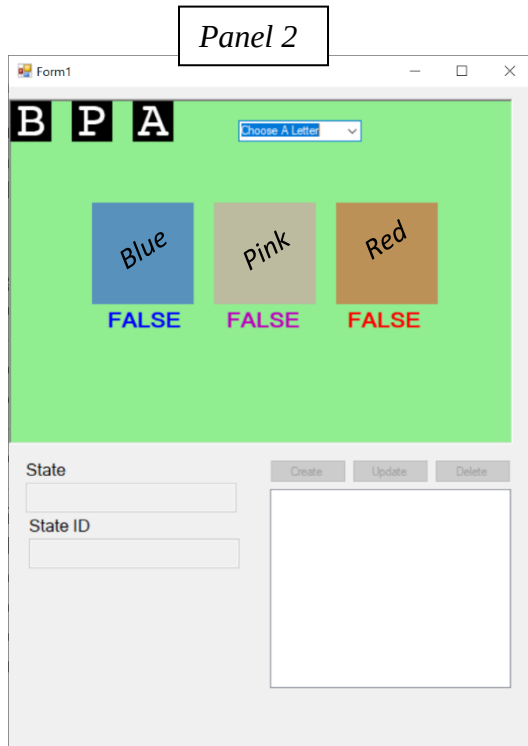
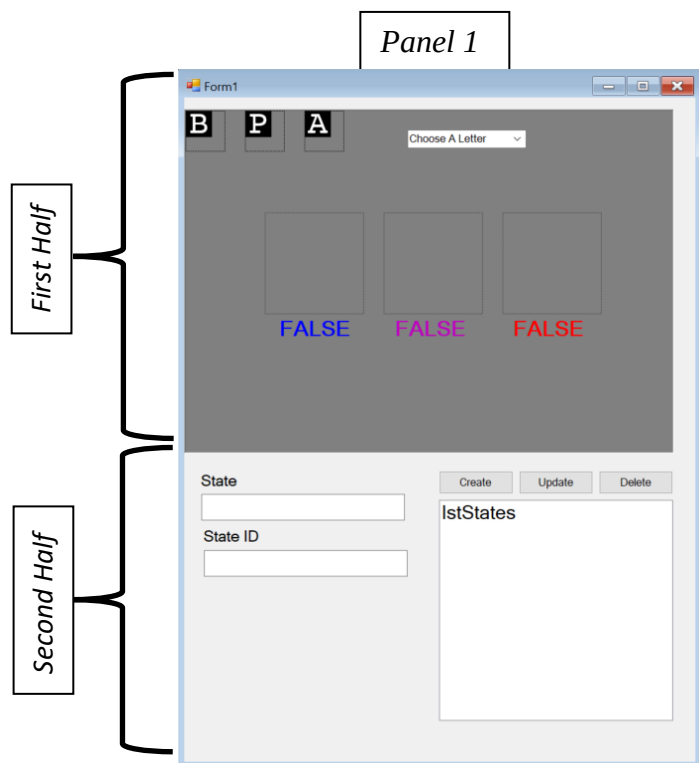
Test Time: 90 minutes

Note To Graders:

- Run the solution code a few times to get a feel for how the app should operate.
- When the program loads there should be a separate form app open. The immediate action should be testing the input of the combo box.
- The second input should be testing to see if the selected letter moves; next test the letters movement on the boundaries of the panel (only test on the first letter one time). Third, test the letter to see if enters target-location for the desired reaction.
- Colors may vary and points should not be deducted unless the student used colors that make any grading impossible or difficult (i.e. they used non contrasting colors).
- Use Input/Output section in the following pages to see how the program should operate or view the solution file.
- Student MUST use a dictionary list for the state information. No exceptions.
- The arrow keys and movement directions must work as intended. No other key inputs will be accepted: only the arrows.
- Comments for the source code documentation must use the naming requirement “SC#” in the comment or there is NO credit. Using “Ctrl+F” in VS will allow you to quickly find all the comments linked to the points in the source code section.
- There will be NO partial credit points for any grading item. All or nothing.
- There are events that may cause the program to crash. You can only grade up to that point. The code may work for a section that did not get a chance to run because of an error in another part, the error cannot be fixed by the grader, hence that portion cannot receive credit. Only items that run can be graded.
- Students can receive credit for code that does not have specific requirements, and their solution uses an approach different than the solution.

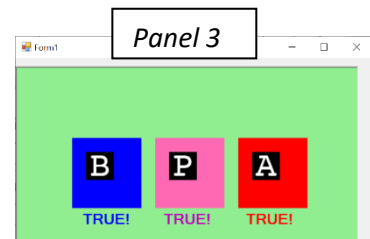
Input/Output

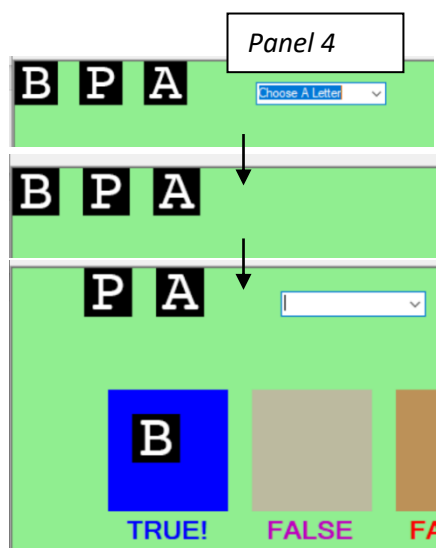
Panel 1 (First Half) displays the first half and second half in the design form. The BPA picture boxes are already designed to be a size of 40x40 pixels and will fill the space when the program runs. It is highly advisable to not alter the locations, sizes of the objects in the provided form. It should be noted that color of the panel is set to a dark gray so it can easily be identified. When the program runs, it should turn to another color. You will be required to use light green for its running color. The labels will change from FALSE to TRUE when the letter-key is in the proper location. The Second Half will be the location where the user creates the dictionary of states (click [“Create”] button) and will display in the list box.



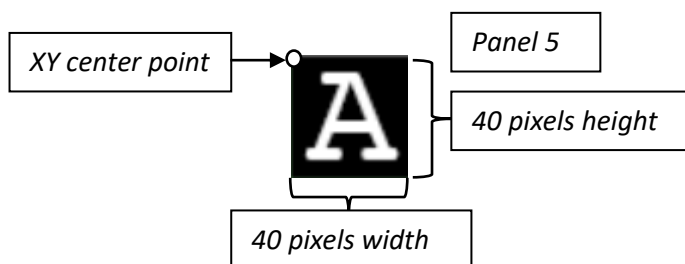
Panel 2 shows the program when it is running. NOTE: due to the probability of receiving a black and white copy of these instructions, the name of the color has been placed on the target-location for you. The alpha channels for these target-locations should be set to 100 in the code you will create. Also notice that all of the object in the second half have been disabled. The dropdown combo box will allow the user to pick which letter they want to move. If they choose ‘B’ then the letter-key that moves until completion will be the ‘B’.

Panel 3 shows the desired end goal of moving the three letters. B targets the blue picture box; P targets the pink picture box; A targets the red picture box. Notice how the labels change from FALSE to TRUE; and notice how the letters are clearly inside of the target and not on the edge.

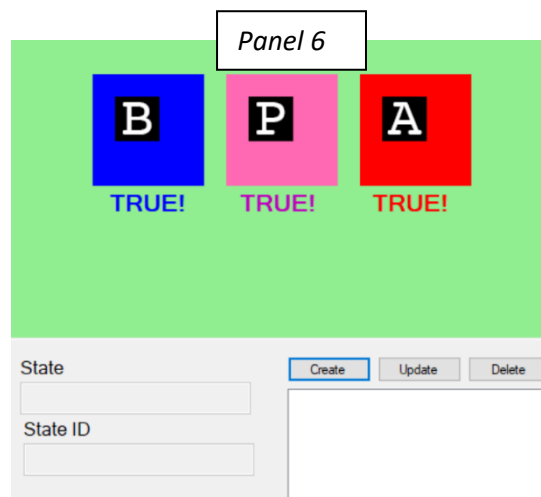




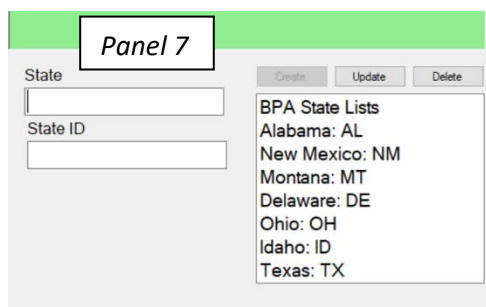
Panel 4 shows the combo box being turned to disabled it is not visible once a letter is selected. In this example ‘B’ is chosen and moved to its target location. The combo box will need to return to being visible and enabled once the letter reaches its target-location. This will need to happen for all letters being selected. Remember, the user can pick the letter-key. When the letters are moving, they will need to move in increments of 20 pixels along the X and Y axis in any direction (up, down, left, and right). The size of the letter-keys are 40x40 pixels and the location of the object is determined by its XY coordinate (**Panel 5**) which is the top left of the shape. The letters cannot move beyond the boundaries of the panel. The formula for movement will need to compensate that the width extends to right and the height extends downward.



Panel 6 brings your attention to the [“Create”] button (second half) and how it is enabled after the targets each have the correct letter-key placed on them. Also, the combo box with letters on the drop-down list from the first half should still be disabled and not visible to avoid any confusion.



Panel 7 demonstrates how the dictionary list is created and prints to the list box. Also notice that the text entry boxes and other buttons are enabled after the list is created. A text header is added to the list “BPA State Lists”.



The [“Create”] button will be used one time. After it creates the dictionary list it will be disabled and never reactivated during the use of the form. In **Panel 7** you can see that is disabled.

Panel 8.a

Panel 8.a demonstrates the Update button. The user enters in their state and abbreviation into the proper fields. Then the ["Update"] button is clicked, and it causes a Message Box to appear (**Panel 8.b**) letting the user know they successfully added the record. Then after the Message Box is closed the new entry is displayed at the bottom of the list box (**Panel 8.c**). NOTE: do not worry about spelling errors or making the dictionary in alphabetic order.

Panel 8.b

Panel 8.c

Panel 9.a demonstrates the Message Box you will program to appear if the user does not enter any alpha-numeric characters into either text box for

creating a new state. It is the same message for both text boxes. **Panel 9.b** shows another Message Box needs to be created to warn the user if they attempt to create a duplicate state. Texas is used in this example.

Panel 9.a

Panel 9.b

Panel 10.a

Panel 10.a shows the process for deleting a record from the dictionary. The user selects the entry, and then hits delete. If successful, a Message Box appears (**Panel 10.b**), and in **Panel 10.c** it can be seen that the selected entry (Alabama in this example) was deleted.

Panel 10.b

Panel 10.c

Panel 11.a is a Message Box that warns the user they did not make a selection for deletion. This will need to be programmed to detect if nothing is selected. **Panel 11.b** is another error that is displayed when the user attempts to delete the dictionary list header "BPA State Lists."

Panel 11.a

Panel 11.b

Solution and Project (There is NO partial credit)		
The project is present on the flash drive		10 points
Program Execution		
The program runs from the USB flash drive: the form loads with no initial errors		10 points
<i>If the program does not execute, then the remaining items in this section receive a score of zero.</i>		
Start Up: the combo box is active and can be accessed		10 points
Start Up: all labels below the target picture boxes are set to false		10 points
Start Up: the picture boxes with the letters BPA are present in that exact order.		10 points
Start Up: all objects from the second half of the program are not active		20 points
First time use: when any of the letters are selected from the combo box it is not visible;		10 points
When 'B' is selected it moves in four directions equally (all or nothing)		10 points
When 'P' is selected it moves in four directions equally (all or nothing)		10 points
When 'A' is selected it moves in four directions equally (all or nothing)		10 points
None of the letters can move off the panel where the letter breaches the edge of the panel (all or nothing)		30 points
Engaging Target: when the letters are clearly inside the target it changes colors (all or nothing)		30 points
Engaging Target: when the letters are clearly inside the target the label changes from FALSE to TRUE (all or nothing)		30 points
Second Half: the "Create" button of program unlocks when the letters are arranged in BPA, in that exact order on the targets.		20 points
Second Half: when ["Create"] button is pressed the list is displayed in the list box in the exact order.		20 points
Second Half: capable to add a state and abbreviation to the list with the ["Update"] button; Message Box must appear.		20 points
Second Half: Message Box displays if trying to enter a duplicate state (program must keep running).		20 points
Second Half: Message Box displays if trying to enter a blank state entry (program must keep running).		20 points
Second Half: can successfully delete an item from the list; Message Box must appear.		20 points
Second Half: Message Box displays if trying to delete with no selection (program must keep running).		20 points
Second Half: Message Box displays if trying to delete BPA header (program must keep running).		20 points
Subtotal		/360 Points

Source Code Review (There is NO partial credit)		
A comment containing the contestant number is present		10 points
Form1.cs class: implements and uses the “controller.cs” interface (four methods total). Code must demonstrate how movement is controlled to prevent movement beyond the boundaries. There also must be code that demonstrates how code is able to get the user key inputs; HINT: you will need to use a separate method for this. Comments are required; mark these comments as “SC1” at the beginning of the four implemented interface methods, plus a comment for the method that that gets the key inputs.		50 points
Form1.cs class: code showing how you set the alpha channels to 100 for the three pictures boxes that are the target-location. All or nothing. Comment is required; mark this comment as “SC2” at the beginning.		10 points
Form1.cs class: <i>cmbLetters_SelectedIndexChanged</i> method is commented explaining how the selection is made and how it is removed. The code must have worked to get credit. Mark this comment as “SC3” at the beginning.		10 points
Form1.cs class: <i>ComboOnOff</i> method code with comments showing how the combobox was enabled/visible and disabled/nonvisible in between selections. The code must have worked to get credit. Mark this comment as “SC4” at the beginning.		30 points
Form1.cs class: <i>Student created method</i> that is used to check if the letter-key is inside of the target-location. The code must have worked to get credit. Mark this comment as “SC5” at the beginning.		30 points
Form1.cs class: <i>Student created method</i> that activates the second half of the program once all three letter-keys have been marked as true in their target-location. The code must have worked to get credit. Mark this comment as “SC6” at the beginning.		10 points
Form1.cs class: <i>HardCodeInfo</i> method demonstrates how hard coded text is entered into a dictionary. The code must have worked to get credit. Mark this comment as “SC7” at the beginning.		10 points
Form1.cs class: <i>btnCreate_Click</i> method is commented explaining how it causes the dictionary records to display on the list box with the proper header “BPA State Lists”. The code must have worked to get credit. Mark this comment as “SC8” at the beginning.		20 points
Form1.cs class: <i>btnUpdate_Click</i> method is commented explaining how the code checks dictionary for duplicate and blank entries. You must use Dictionary method <i>Contains</i> . The code must have worked to get credit. Mark this comment as “SC9” at the beginning.		20 points
Form1.cs class: <i>btnDelete_Click</i> method is commented explaining how the code deals with NullReferenceException and ArgumentOutOfRangeException. Code must use at least 1 try/catch. The code must have worked to get credit. Mark this comment as “SC10” at the beginning.		40 points
Subtotal		/240 Points
Total Points		/600 Points

```
...BPA2022\National\NationalBPA_KEY\NationalBPA\Form1.cs 1
1 using System;
2 using System.Collections.Generic;
3 using System.ComponentModel;
4 using System.Data;
5 using System.Drawing;
6 using System.Linq;
7 using System.Text;
8 using System.Threading.Tasks;
9 using System.Windows.Forms;
10 using System.Collections;
11 using System.Collections.Specialized;
12
13 namespace NationalBPA
14 {
15     public partial class Form1 : Form, Controller
16     {
17
18         public bool onOff;
19         public bool canMove;
20         public int counter = 0;
21         public PictureBox pbSelect; //The PB that will be moving
22         public PictureBox pbTarget; //The PB target we want to move
23         public Label lblTarget; //Changes from False to True
24
25         ListDictionary stateDictionary = new ListDictionary();
26
27         public Form1()
28         {
29             InitializeComponent();
30         }
31
32         private void Form1_Load(object sender, EventArgs e)
33         {
34             cmbLetters.Enabled = true; //Allows the combo box to be
35             onOff = true; //Used to toggle the combo box being visible and
36             canMove = true;
37             //Original back color is transparent. (Alpha [0 to 255], RGB)
38             //Students will be required to set Alpha Channel
39             //SC2
40             pb_Target_A.BackColor = Color.FromArgb(100, Color.Red);
41             pb_Target_P.BackColor = Color.FromArgb(100, Color.HotPink);
42             pb_Target_B.BackColor = Color.FromArgb(100, Color.Blue);
43             //Send the targets (Keys) to backmost layer
44             pb_Target_A.SendToBack(); pb_Target_B.SendToBack();
45             pb_Target_P.SendToBack();
```

```
...BPA2022\National\NationalBPA_KEY\NationalBPA\Form1.cs 2
46     panel1.BackColor = Color.LightGreen;
47     //Text boxes, lists, and buttons have to be disabled or the
48     //picture box cotrols will not work.
49     txtID.Enabled = false; txtStateName.Enabled = false;
50     btnCreate.Enabled = false; btnDelete.Enabled = false;
51     btnUpdate.Enabled = false;
52     lstStates.Enabled = false;
53 }
54
55 //Interface implementation
56 //NOTE picturebox has its XY coordinate center in the top left.
57 //SC1 1 of 5
58 public void MoveUp(PictureBox pb, bool canMove)
59 {
60     Point pos = pb.Location;
61     if (pos.Y > 0 && canMove)
62     {
63         pos.Y -= 20;
64
65     }
66     pb.Location = pos;
67     TargetCheck(); //Checking to see if we are in the target
68 }
69
70 //SC1 2 of 5
71 public void MoveDown(PictureBox pb, bool canMove)
72 {
73     Point pos = pb.Location;
74     //This subtraction of the pb.Height keeps the picture box in
75     //the panel
76     if (pos.Y < panel1.Height - pb.Height && canMove)
77     {
78         pos.Y += 20;
79
80     }
81     pb.Location = pos;
82     TargetCheck();
83 }
84 //SC1 3 of 5
85 public void MoveRight(PictureBox pb, bool canMove)
86 {
87     Point pos = pb.Location;
88     //This subtraction of the pb.Width keeps the picture box in the
89     //panel
90     if (pos.X < panel1.Width - (pb.Width+10) && canMove)
91     {
92         pos.X += 20;
```

```
...BPA2022\National\NationalBPA_KEY\NationalBPA\Form1.cs 3
92         }
93         pb.Location = pos;
94         TargetCheck();
95     }
96     //SC1 4 of 5
97     public void MoveLeft(PictureBox pb, bool canMove )
98     {
99         Point pos = pb.Location;
100         if (pos.X > 0 && canMove)
101         {
102             pos.X -= 20;
103         }
104         pb.Location = pos;
105         TargetCheck();
106     }
107
108     //Form driven key events that call on the methods from the Controller interface
109     //Not given but they will have to use it
110     //SC1 5 of 5
111     private void Form1_KeyDown(object sender, KeyEventArgs e)
112     {
113         if (e.KeyCode == Keys.Up)
114         {
115             MoveUp(pbSelect, canMove);
116         }
117         if (e.KeyCode == Keys.Down)
118         {
119             MoveDown(pbSelect, canMove);
120         }
121         if (e.KeyCode == Keys.Right)
122         {
123             MoveRight(pbSelect, canMove);
124         }
125         if (e.KeyCode == Keys.Left)
126         {
127             MoveLeft(pbSelect, canMove);
128         }
129     }
130     //Combo box implementation
131     //SC3
132     private void cmbLetters_SelectedIndexChanged(object sender, EventArgs e)
133     {
134         string selection = cmbLetters.SelectedItem.ToString();
135
136         Console.WriteLine(selection); //DELETE ME
137         if(selection == "Letter A")
138         {
```

```
...BPA2022\National\NationalBPA_KEY\NationalBPA\Form1.cs 4
139
140         pbSelect = pb_letter_A; //Sets up the program to know  ➤
           which letter to move
141         pbTarget = pb_Target_A; //Sets up which color target to  ➤
           aim for
142         lblTarget = lbl_A_Key; //Sets up which label will change
143     }
144     else if(selection == "Letter B")
145     {
146         pbSelect = pb_letter_B;
147         pbTarget = pb_Target_B;
148         lblTarget = lbl_B_Key;
149     }
150     else if(selection == "Letter P")
151     {
152         pbSelect = pb_letter_P;
153         pbTarget = pb_Target_P;
154         lblTarget = lbl_P_Key;
155     }
156     cmbLetters.Items.Remove(selection); //Removed the item options
157     ComboOnOff();
158 }
159
160 //Toggles the Combo Box.
161 //Students may do something different.
162 //This will be SC4
163 private void ComboOnOff()
164 {
165     onOff = !onOff;
166     cmbLetters.Enabled = onOff;
167     if(counter >=3)
168     {
169         cmbLetters.Visible = false; //Kill switch to make sure it  ➤
           turns off
170     } else
171     {
172         cmbLetters.Visible = onOff;
173     }
174 }
175 }
176
177
178 //This is SC5 suggestion. //Students results will be different  ➤
           since no method provided
179 private bool TargetCheck()
180 {
181     //Checks to make sure the BPA letters clear the edge of the  ➤
           target
182     //The calculation has to take into account the width/height of  ➤
```

```
...BPA2022\National\NationalBPA_KEY\NationalBPA\Form1.cs 5
    the moving shape
183    //and the width/height of the target. PB's XY coordinates are  ➤
    the top left of the
184    //shape. Y increases going down.
185    if (pbSelect.Location.X > pbTarget.Location.X &&  ➤
        pbSelect.Location.X < pbTarget.Location.X+ pbTarget.Width -  ➤
        pbSelect.Width && pbSelect.Location.Y > pbTarget.Location.Y  ➤
        && pbSelect.Location.Y < pbTarget.Location.Y +  ➤
        pbTarget.Height - pbSelect.Height)
186    {
187        counter++; //Keeping track of how many targets have been  ➤
        completed
188
189        //Give the user two indicators they hit the target by  ➤
        changing label text
190        //and changing the color alpha channel to 100%(255) opaque
191        pbTarget.BackColor = Color.FromArgb(255,  ➤
        pbTarget.BackColor);
192        lblTarget.Text = "TRUE!";
193
194        ComboOnOff();
195        cmbLetters.Focus();
196        if (counter >= 3) //Three is the max. This could have been  ➤
            "=="
197        {
198            canMove = false; //a kill switch on the controls.
199            EnableDB();
200            return true;
201        }
202        else
203        {
204            return false;
205        }
206    }
207    return false;
208
209
210
211
212 }
213
214
215
216 //SC6 suggestion. //Students results will be different no method  ➤
    provided.
217 private void EnableDB()
218 {
219
220     btnCreate.Enabled = true; btnDelete.Enabled = true;  ➤
```

```
...BPA2022\National\NationalBPA_KEY\NationalBPA\Form1.cs 6
    btnUpdate.Enabled = true;
221    lstStates.Enabled = true;
222    btnCreate.Focus();
223
224
225    }
226
227    //We will be adding student home state and BPA ID; these are hard ➤
    coded entries
228    //Students will get the string information. The buttons will allow ➤
    additional entries.
229    //SC7
230    private void HardCodeInfo()
231    {
232        stateDictionary.Add("Alabama", "AL");
233        stateDictionary.Add("New Mexico", "NM");
234        stateDictionary.Add("Montana", "MT");
235        stateDictionary.Add("Delaware", "DE");
236        stateDictionary.Add("Ohio", "OH");
237        stateDictionary.Add("Idaho", "ID");
238        stateDictionary.Add("Texas", "TX");
239
240
241    }
242
243    //This buttons adds the records to the dictionary. It can only run ➤
    once!
244    //Students need to make sure it does not run more than once which ➤
    will
245    //create errors.
246    //SC8 1 of 2 (see populateList() below)
247    private void btnCreate_Click(object sender, EventArgs e)
248    {
249        HardCodeInfo();
250        btnCreate.Enabled = false;
251        txtID.Enabled = true; txtStateName.Enabled = true;
252        txtStateName.Focus();
253        populateList();
254
255
256    }
257    //Students do not have to use this method by traversing the ➤
    Dictionary.
258    //This method is used to clear and populate the list for all ➤
    buttons
259    //Will need to mention some of this code (adding to dictionary and ➤
    header)
260    //SC8 2 of 2
261    private void populateList()
```

```
...BPA2022\National\NationalBPA_KEY\NationalBPA\Form1.cs 7
262     {
263         lstStates.Items.Clear();
264         lstStates.Items.Add("BPA State Lists"); //header
265         foreach (DictionaryEntry entry in stateDictionary) //adds to the dictionary ➤
266         {
267             lstStates.Items.Add(entry.Key + ": " + entry.Value);
268         }
269     }
270     //This adds the entry to the dictionary. Simple string checks
271     //can determine if information is entered.
272     //SC9
273     private void btnUpdate_Click(object sender, EventArgs e)
274     {
275         string idTemp = txtID.Text;
276         string nameTemp = txtStateName.Text;
277         if (stateDictionary.Contains(nameTemp))
278         {
279             MessageBox.Show("This state already exists!");
280             txtStateName.Text = "";
281             txtID.Text = "";
282             txtStateName.Focus();
283         }
284         else if (txtStateName.Text == "" || txtID.Text == "")
285         {
286             MessageBox.Show("You are missing data entry!");
287             txtStateName.Text = "";
288         }
289         else
290         {
291             stateDictionary.Add(nameTemp, idTemp);
292             MessageBox.Show("Your entry was successful\n" + "State:\t" ➤
293                             + nameTemp + "\nID:\t" + idTemp);
294             txtID.Text = ""; txtStateName.Text = "";
295         }
296         populateList();
297     }
298     //This buttons deletes the selected value. There are multiple ways ➤
299     //to accomplish this
300     //In this code we just check to see if there is a colon since that ➤
301     //is added to
302     //the listed records. NOTE: students will need to solve how to ➤
303     //ensure there is no
304     //NullReferenceException(nothing is selected) and
305     //ArgumentOutOfRangeException (thrown when they select a non ➤
306     //dictionary record
307     //which is BPA in this case.
308     //SC10
```

```
...BPA2022\National\NationalBPA_KEY\NationalBPA\Form1.cs 8
305     private void btnDelete_Click(object sender, EventArgs e)
306     {
307         try
308         {
309             string tempDEL;
310             if (lstStates.SelectedItem != null)
311             { tempDEL = lstStates.SelectedItem.ToString();
312               tempDEL = tempDEL.Substring(0, tempDEL.IndexOf(":"));
313               if (stateDictionary.Contains(tempDEL))
314               {
315                   stateDictionary.Remove(tempDEL);
316                   populateList();
317                   MessageBox.Show("Deletion was successful");
318               }
319               else
320               {
321                   MessageBox.Show("This action cannot be performed  ➤
322                     on BPA.");
323               }
324             }
325             else
326             {
327                 MessageBox.Show("You did not make a selection.");
328             }
329         }
330     }
331     catch (ArgumentOutOfRangeException ➤
332           argumentOutOfRangeException)
333     {
334         //MessageBox.Show($"Error: ➤
335           {argumentOutOfRangeException.Message}");
336         MessageBox.Show($"Error: This action cannot be performed ➤
337           on BPA Header.");
338     }
339 }
340
341 }
342
343 }
344
```

```
...22\National\NationalBPA_KEY\NationalBPA\Controller.cs 1
1 using System;
2 using System.Collections.Generic;
3 using System.Linq;
4 using System.Text;
5 using System.Threading.Tasks;
6 using System.Windows.Forms;
7
8 namespace NationalBPA
9 {
10     internal interface Controller
11     {
12         void MoveUp(PictureBox pb, bool cm);
13         void MoveDown(PictureBox pb, bool cm);
14         void MoveLeft(PictureBox pb, bool cm);
15         void MoveRight(PictureBox pb, bool cm);
16     }
17 }
18
```